Auto Scaling

Important
You must have completed the following chapters as pre-requisites for this lab:

- Fundamentals

Since a single Fargate instance corresponds to a single ECS task, you need to specify task's CPU and memory during creating task definition. Therefore, it's crucial to right-size your Fargate tasks to ensure they can perform their duties with the desired performance level. If a task struggles due to insufficient CPU or memory for performing its functions, this indicates that the task is not correctly sized and might require additional resources. You can accurately assess the needs of your application by engaging in performance measurement, conducting comprehensive load testing, or closely observing key metrics.

Once you're confident that your tasks are appropriately sized, you can then scale horizontally by deploying additional tasks to handle more requests. Horizontal scaling is the preferred method for scaling cloud-native, containerized workloads.

Amazon ECS offers the capability to automatically adjust the desired number of tasks within your service, a feature known as **Service Auto Scaling**. ECS Service Auto Scaling uses **Application Auto Scaling** to provide this functionality. For auto scaling to be effective, the metric used must be a **proportional metric**. By proportional, we mean it must exhibit a particular behavior: if we hold the number of tasks in our service steady and the load doubles, then the value of the metric should double as well.

In the context of ECS when utilizing EC2 instances, it's also important to consider the use of capacity providers for managing the capacity of EC2 instances in addition to ECS Service Auto Scaling. However, this chapter focuses primarily on Fargate, so we will not be covering ECS capacity providers in detail.

Service Auto Scaling

Amazon ECS seamlessly integrates with **Amazon CloudWatch** to enable efficient scaling of ECS services based on real-time metrics. These metrics are transmitted from Amazon ECS to CloudWatch at one-minute intervals, allowing for precise monitoring and timely scaling decisions. When metrics exceed the thresholds defined in your scaling policy, CloudWatch triggers an alarm that adjusts the desired number of tasks within your service. This dynamic adjustment process increases the desired capacity during scale-out events and decreases it during scale-in events, ensuring optimal resource utilization.

Amazon ECS offers three sophisticated service scaling strategies:

1. **Target Tracking Scaling**: This method aims to maintain a specified scaling metric at a target value by automatically adjusting the number of tasks. Target tracking scaling is preferred for its simplicity and low maintenance requirements, making it an ideal choice for businesses seeking operational efficiency without constant manual intervention.
2. **Step Scaling**: This strategy provides greater control over scaling actions. Users can select metrics, set threshold values, and define step adjustments to specify the number of resources to add or remove. It also allows for customizable breach evaluation periods for metric alarms, offering a tailored approach to handling variable workloads effectively.
3. **Scheduled Scaling**: This method is best utilized when scaling actions can be anticipated based on known demand patterns. It's ideal for applications experiencing predictable traffic fluctuations, enabling proactive resource management to ensure service stability and performance during peak times.

These scaling methods empower organizations to harness the full potential of ECS, optimizing both cost and performance by aligning resource allocation with actual demand. By leveraging these strategies, businesses can ensure their applications remain responsive and efficient, regardless of fluctuations in workload or user demand.

Previous Next

Target Tracking Scaling

In this section, we'll configure ECS Service Auto Scaling using Target Tracking Scaling. This includes determining which services to set up application autoscaling for and applying the appropriate scaling policies.

Let's register the UI service as a scalable target with Application Auto Scaling. The following command sets the scaling range for the UI Service from a minimum of 2 to a maximum of 10 tasks.

```
aws application-autoscaling register-scalable-target \
    --service-namespace ecs \
    --scalable-dimension ecs:service:DesiredCount \
    --resource-id service/retail-store-ecs-cluster/ui \
    --min-capacity 2 \
    --max-capacity 10
```

Next, we'll create a scaling policy for our scaling target.

First, create a JSON configuration file for the scaling policy. This configuration utilizes the predefined metric type of **request count per target** related to the Application Load Balancer that routes requests to the ECS service. In this case, we're aiming for 1,500 requests per ECS task (or target).

> This scaling policy is only an example. You should understand the scaling profile of your particular workloads to determine the appropriate scaling metrics and thresholds before enabling autoscaling.

```
cat << EOF > ui-scaling-policy.json
{
    "TargetValue": 1500,
    "PredefinedMetricSpecification": {
        "PredefinedMetricType": "ALBRequestCountPerTarget",
        "ResourceLabel": "$UI_ALB_PREFIX/$UI_TG_PREFIX"
    }
}
EOF
```

Note that we're using the `$UI_ALB_PREFIX` and `$UI_TG_PREFIX` environment variables to specify the correct ALB and Target Group for our UI service.

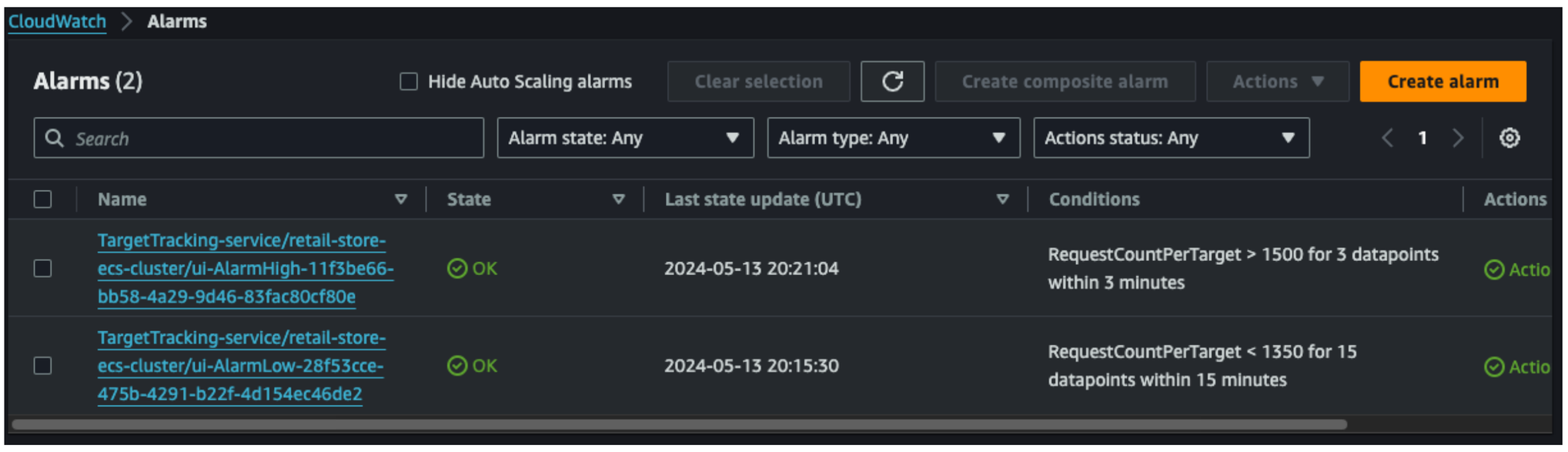
Now, apply the scaling policy with the following command:

```
aws application-autoscaling put-scaling-policy \
    --service-namespace ecs \
    --scalable-dimension ecs:service:DesiredCount \
    --resource-id service/retail-store-ecs-cluster/ui \
    --policy-name ui-scaling-policy --policy-type TargetTrackingScaling \
    --target-tracking-scaling-policy-configuration file://ui-scaling-policy.json
```

Explore CloudWatch Alarm

When you navigate to the **Alarms** tab in the CloudWatch service, you will see that the scaling policy has created 2 CloudWatch alarms.

> The alarm status may vary depending on request numbers. For example, `UI-AlarmLow` triggers when requests fall below `1350`.



High Alarm (Scale-out) configuration:

- **Metric**: `ALBRequestCountPerTarget` > 1500
- **Evaluation**: 3 data points within 3 minutes
- **Action**: When in ALARM state, adds more tasks to the ECS service
- **Behavior**: Continues adding tasks in subsequent evaluation periods if the alarm persists, up to the maximum specified in the scaling policy

Low Alarm (Scale-in) configuration:

- **Metric**: `ALBRequestCountPerTarget` < 1350
- **Evaluation**: 15 data points within 15 minutes
- **Action**: When in ALARM state, reduces the number of tasks
- **Behavior**: Scales in more slowly to prioritize high availability

Trigger Auto Scaling

In this section, we'll generate synthetic load for our `UI` service to observe its scaling behavior.

First, verify the DNS name for the load balancer associated with our UI service. This environment variable should have been exported as part of the fundamental chapter.

```
export RETAIL_ALB=$(aws elbv2 describe-load-balancers --name retail-store-ecs-ui \
  --query 'LoadBalancers[0].DNSName' --output text)
echo ${RETAIL_ALB}
```

Next, we'll use the hey tool to send requests to the `/home` path of the UI service:

```
hey -n 1000000 -c 5 -q 40 http://$RETAIL_ALB/home &
```

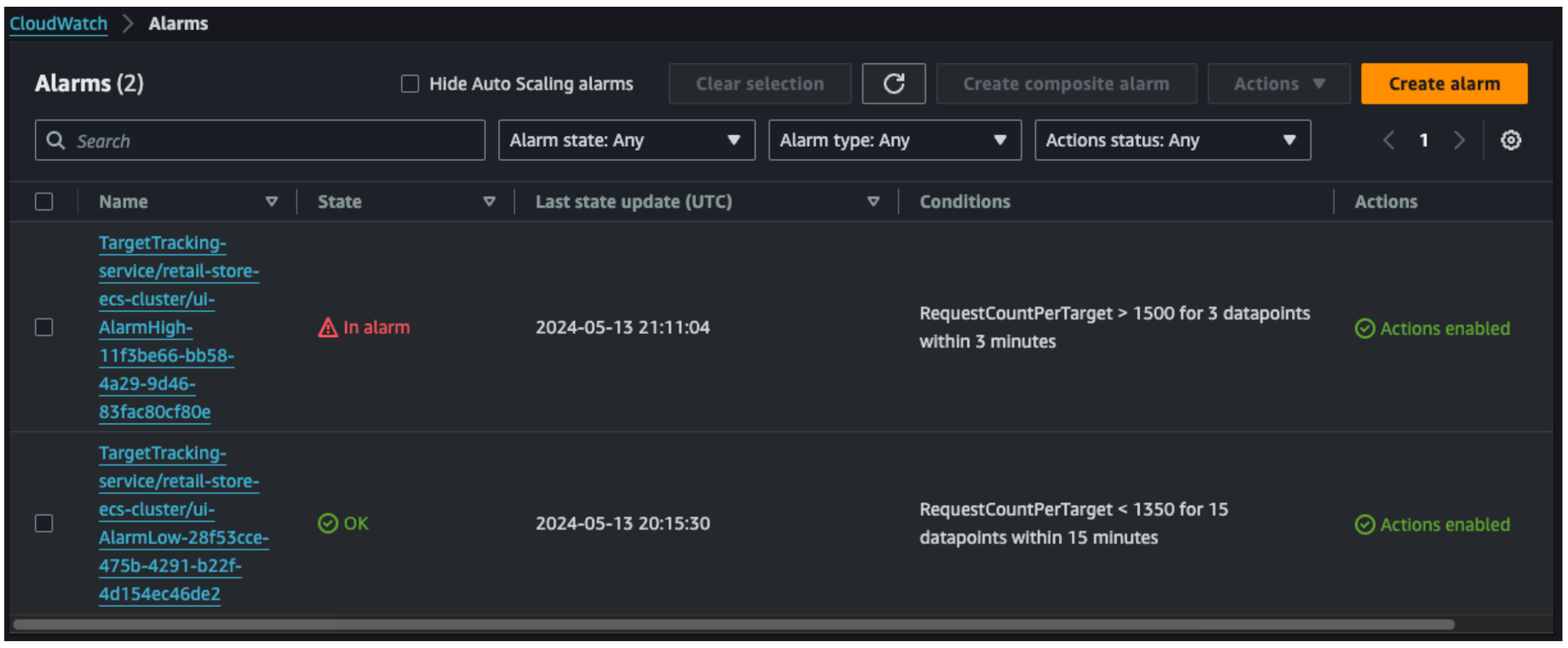
Scaling activity will be triggered when the **high alarm** for the scaling metric breaches for 3 consecutive 1-minute periods. If you want to automatically wait until the alarm triggers, you can run this command (**~ 4 min**):

```
sleep 90 && aws cloudwatch wait alarm-exists --alarm-name-prefix \
 TargetTracking-service/retail-store-ecs-cluster/ui-AlarmHigh --state-value ALARM
```

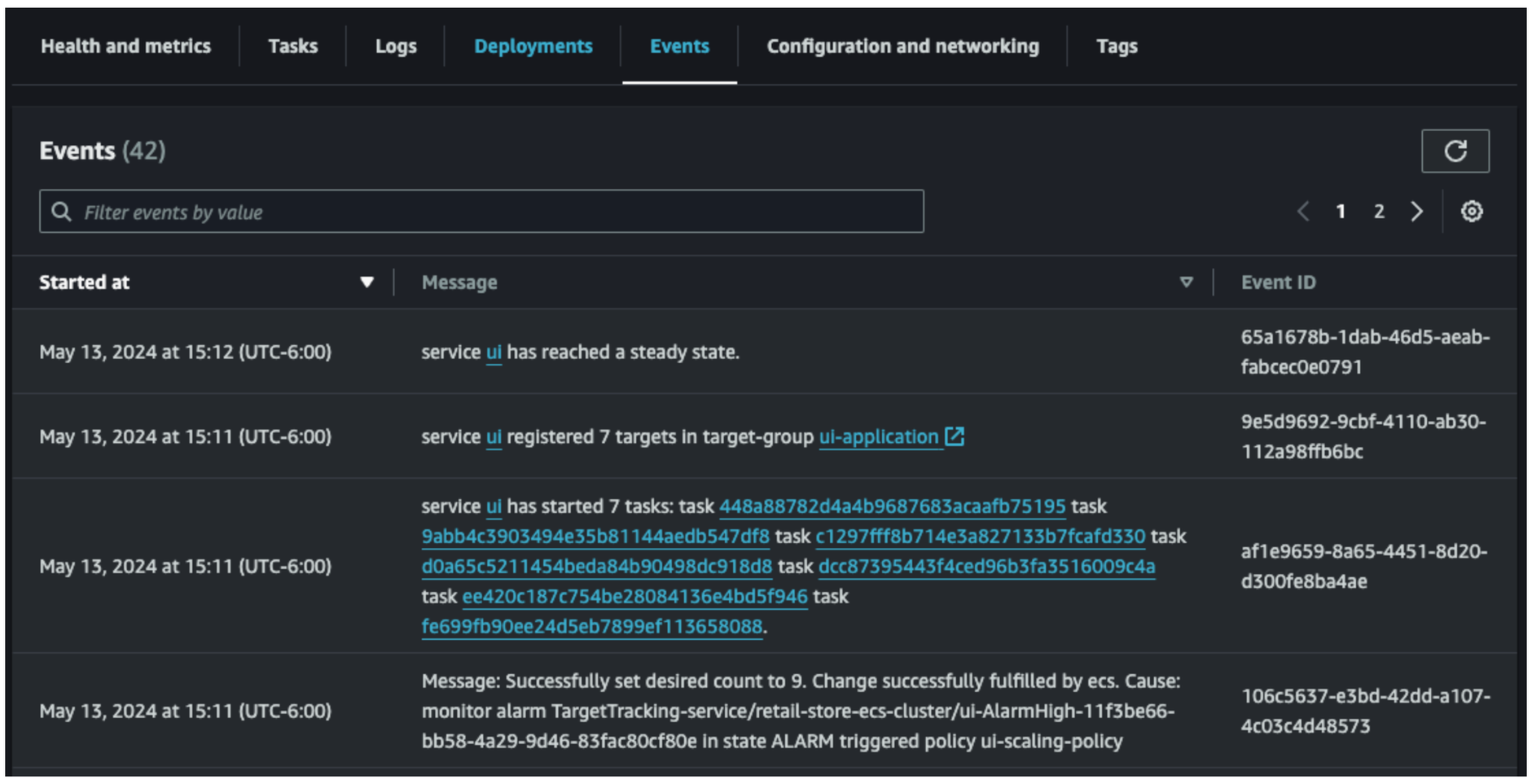
When the alarm fires, you'll notice the service's task count scaling out from 2 to a higher number:

```
aws ecs describe-tasks \
    --cluster retail-store-ecs-cluster \
    --tasks $(aws ecs list-tasks --cluster retail-store-ecs-cluster --query 'taskArns[]' --output text) \
    --query "tasks[*].[group, launchType, lastStatus, healthStatus, taskArn]" --output table
```

You can observe the associated high alarm with the scaling policy transitioning to the **ALARM** state in the CloudWatch console, as shown below.



You can also check the Events tab in the `UI` Service page to see the scaling activity, where the desired count increases beyond the initial task count.



To stop the `hey` process, run the following command:

```
pkill -9 hey
```

Normally, after several minutes, the number of tasks should scale back to the minimum of 2. However, for the sake of time, we can force the service to scale down by running the following commands

```
aws ecs wait services-stable --cluster retail-store-ecs-cluster --services ui

aws ecs update-service \
    --cluster retail-store-ecs-cluster \
    --service ui \
    --task-definition retail-store-ecs-ui \
    --desired-count 2
```

Scheduled Scaling

Important
You must have completed the following chapters as pre-requisites for this lab:

- Target Tracking Scaling

In this section, we'll configure ECS Service Auto Scaling using Scheduled Scaling. This includes setting up time-based scaling rules for services that have predictable load patterns. We'll create a scheduled scaling action for the **UI** service using a cron expression to test its scale-out and scale-in by adjusting the minimum value.

The following example is intended for testing purpose and will execute 2 minutes after the command is run. In a production environment, you need to establish schedules for consistent, predictable patterns.

Since we've already covered registering the UI service as a scalable target with Application Auto Scaling in the **Target Tracking section**, we'll skip this part. Let's run the following command to **increase the minimum capacity from 2 to 5** in 2 minutes from the current time:

```
aws application-autoscaling put-scheduled-action \
 --service-namespace ecs \
 --scalable-dimension ecs:service:DesiredCount \
 --resource-id service/retail-store-ecs-cluster/ui \
 --scheduled-action-name "test-scale-up" \
 --schedule "at($(date -u -d "+2 minutes" "+%Y-%m-%dT%H:%M:%S"))" \
 --scalable-target-action MinCapacity=5,MaxCapacity=10
```

To verify the scheduled action, we'll set up a 150 second countdown timer and run the command to check the scheduled action. Use the following command to start the countdown timer:

```
echo "Starting timer for 2 plus minutes..." && sleep 150 && echo "Time's up! and you can run the next command"
```

We will wait for 150 seconds for the scheduled action to trigger and deploy instances. After the timer completes, execute the following command to observe the scheduled action. You can observe list of 5 ECS tasks.

```
aws ecs describe-tasks \
 --cluster retail-store-ecs-cluster \
 --tasks $(aws ecs list-tasks --cluster retail-store-ecs-cluster --service-name ui --query 'taskArns[]' --output text) \
 --query "tasks[*].[group, launchType, lastStatus, healthStatus, taskArn]" --output table
```

Let's run the following command to **scale back** in 2 minutes from current time. This command will **set the minimum capacity from 5 to 2**, which is the original capacity:

```
aws application-autoscaling put-scheduled-action \
 --service-namespace ecs \
 --scalable-dimension ecs:service:DesiredCount \
 --resource-id service/retail-store-ecs-cluster/ui \
 --scheduled-action-name "test-scale-in" \
 --schedule "at($(date -u -d "+2 minutes" "+%Y-%m-%dT%H:%M:%S"))" \
 --scalable-target-action MinCapacity=2,MaxCapacity=10
```

Implementing Real-World Scheduled Scaling

As implemented above, the commands can be adjusted to schedule scaling during the necessary time windows based on your application's requirements. For example, scheduled scaling can be set to trigger at 9 AM UTC and 6 PM UTC each day, which may align with peak usage times for a global application. Here's how you can configure these daily scheduled scaling actions.

The following example can be applied in real life, but there is a high probability that it will not execute during the workshop due to time zone differences.

The following command initiates the scale-out activity, raising the number of tasks to 5 every day at 9:00 AM UTC to accommodate anticipated increased traffic during the day:

```
aws application-autoscaling put-scheduled-action \
    --service-namespace ecs \
    --scalable-dimension ecs:service:DesiredCount \
    --resource-id service/retail-store-ecs-cluster/ui \
    --scheduled-action-name "week-day-scale-out" \
    --schedule "cron(0 9 ? * MON-FRI *)" \
    --scalable-target-action MinCapacity=5,MaxCapacity=10
```

The following command initiates the scale-in activity, reducing the number of tasks during off-peak hours. For instance, we will scale down our UI service at 6 PM each day.

```
aws application-autoscaling put-scheduled-action \
    --service-namespace ecs \
    --scalable-dimension ecs:service:DesiredCount \
    --resource-id service/retail-store-ecs-cluster/ui \
    --scheduled-action-name "week-day-scale-in" \
    --schedule "cron(0 18 ? * MON-FRI *)" \
    --scalable-target-action MinCapacity=2,MaxCapacity=10
```